

Программирование на С

Лекция 13

Пользовательские типы

Зачем нужны структуры?

До этого момента мы научились хранить:

- отдельные значения;
- массивы однотипных значений;
- строки.

Но достаточно ли этого для описания **реальных объектов**?

Пример задачи

Научная установка получает результаты измерений.

Для каждого измерения необходимо сохранить:

- значение;
- погрешность;
- время получения;
- номер датчика.

Например:

Значение:	12.4
Погрешность:	0.2
Время:	1722258000
Датчик:	17

Одно измерение

Для одного измерения можно использовать обычные переменные.

```
double value = 12.4;  
double error = 0.2;  
long timestamp = 1722258000;  
int sensor_id = 17;
```

Пока всё выглядит вполне нормально.

А если измерений много?

Можно сделать так:

```
double value1 = 12.4;  
double error1 = 0.2;  
long timestamp1 = 1722258000;  
int sensor_id1 = 17;  
  
double value2 = 8.7;  
double error2 = 0.1;  
long timestamp2 = 1722258060;  
int sensor_id2 = 4;
```

Очевидно, что такой подход не масштабируется.

Первое естественное решение

Используем массивы.

```
double values[1000];  
double errors[1000];  
long timestamps[1000];  
int sensor_ids[1000];
```

Теперь каждое измерение определяется **общим индексом**.

Где находится одно измерение?

Измерение с индексом `i` состоит из четырёх элементов.

```
values[i]  
errors[i]  
timestamps[i]  
sensor_ids[i]
```

⚠ Самого объекта «**измерение**» пока не существует.

Есть только четыре независимых массива.

Как вывести одно измерение?

```
printf("Value:  %f\n", values[i]);  
printf("Error:  %f\n", errors[i]);  
printf("Time:   %ld\n", timestamps[i]);  
printf("Sensor: %d\n", sensor_ids[i]);
```

Все характеристики приходится получать из разных массивов.

Как передать измерение в функцию?

```
void print_measurement(  
    double value,  
    double error,  
    long timestamp,  
    int sensor_id  
);
```

Вызов:

```
print_measurement(  
    values[i],  
    errors[i],  
    timestamps[i],  
    sensor_ids[i]
```

Сортировка измерений

Необходимо отсортировать измерения по времени.

Нужно поменять местами элементы **во всех массивах**.

```
values  
errors  
timestamps  
sensor_ids
```

Если забыть хотя бы один массив — данные перестанут соответствовать друг другу.

Что произойдёт?

Предположим, мы отсортировали только время.

```
swap(timestamps[i], timestamps[j]);
```

Получится примерно следующее:

Значение	Погрешность	Время	Датчик
12.4	0.2	время другого измерения	17

⚠ Одно измерение «развалилось».

Добавим ещё одну характеристику

Например:

- качество сигнала;
- температура;
- координаты датчика.

Теперь необходимо изменить:

- хранение данных;
- ввод;
- вывод;
- сортировку;
- копирование;
- все функции.

Чем больше характеристик, тем сложнее поддерживать программу.

Что нам действительно нужно?

Хотелось бы работать не с четырьмя массивами,
а с **одним объектом**.

Measurement

```
|— value
|— error
|— timestamp
└— sensor_id
```

Тогда можно было бы писать:

- одно измерение;
- массив измерений;

Возникает вопрос

Массив объединяет **много значений одного типа**

А можно ли создать новый тип данных, который объединяет **несколько связанных значений разных типов?**

Именно такую возможность предоставляет язык **C**.

Структура

```
struct Measurement
{
    double value;
    double error;
    long timestamp;
    int sensor_id;
};
```

Так мы описываем новый тип данных:
измерение.

Что описывает структура?

```
struct Measurement
{
    double value;
    double error;
    long timestamp;
    int sensor_id;
};
```

Measurement

- └ value
- └ error
- └ timestamp
- └ sensor_id

Определение

Структура — это составной тип данных, объединяющий несколько именованных значений.

Эти значения называются **полями структуры**.

Поля структуры могут иметь разные типы.

Массив и структура

Массив

```
double values[1000];
```

- много элементов;
- все одного типа;
- доступ по индексу.

Структура

```
struct Measurement  
{  
    double value;  
    double error;  
    long timestamp;  
    int sensor_id;  
};
```

- несколько полей;
- типы могут различаться;
- доступ по имени.

Синтаксис объявления структуры

```
struct Measurement
{
    double value;
    double error;
    long timestamp;
    int sensor_id;
};
```

- `struct` — ключевое слово;
- `Measurement` — тег структуры;
- внутри `{ }` перечислены поля;
- после `}` обязательно ставится `;`.

Объявление типа не создаёт объект

```
struct Measurement
{
    double value;
    double error;
    long timestamp;
    int sensor_id;
};
```

После этого ещё не существует ни одного измерения.

Мы только описали,
как будет устроена переменная такого типа.

Создание экземпляра структуры

```
struct Measurement measurement;
```

Теперь появилась переменная `measurement`.

Она содержит все поля,
описанные в `struct Measurement`.

```
measurement
├── value
├── error
├── timestamp
└── sensor_id
```

Несколько экземпляров

```
struct Measurement first;  
struct Measurement second;
```

Обе переменные имеют один тип,
но хранят независимые значения.

```
first  
├── value  
├── error  
├── timestamp  
└── sensor_id
```

```
second  
├── value  
├── error  
├── timestamp  
└── sensor_id
```

Доступ к полям

Для обращения к полю используется оператор `.`.

```
measurement.value  
measurement.error  
measurement.timestamp  
measurement.sensor_id
```

Общая форма:

```
имя_структуры.имя_поля
```

Заполнение полей

```
struct Measurement measurement;  
  
measurement.value = 12.4;  
measurement.error = 0.2;  
measurement.timestamp = 1722258000;  
measurement.sensor_id = 17;
```

Каждое поле ведёт себя как обычная переменная своего типа.

Использование полей

```
measurement.value += 1.0;

if (measurement.error > 0.5)
{
    printf("Low accuracy\n");
}
```

```
printf("Value:  %.1f\n", measurement.value);
printf("Error:  %.1f\n", measurement.error);
printf("Time:    %ld\n", measurement.timestamp);
printf("Sensor:  %d\n", measurement.sensor_id);
```

Неинициализированные поля

```
int main(void)
{
    struct Measurement measurement;

    printf("%f\n", measurement.value);
}
```

Поля локальной структуры без инициализации имеют неопределённые значения.

Перед чтением поля необходимо задать ему значение.

Позиционная инициализация

```
struct Measurement measurement =  
{  
    12.4,  
    0.2,  
    1722258000,  
    17  
};
```

Значения сопоставляются полям в порядке их объявления.

Порядок имеет значение

```
struct Measurement
{
    double value;        // 12.4
    double error;        // 0.2
    long timestamp;      // 1722258000
    int sensor_id;       // 17
};
```

При позиционной инициализации легко ошибиться,
если полей много.

Назначенные инициализаторы

```
struct Measurement measurement =  
{  
    .value = 12.4,  
    .error = 0.2,  
    .timestamp = 1722258000,  
    .sensor_id = 17  
};
```

Такой код легче читать:

сразу видно, какому полю соответствует значение.

Порядок можно менять

```
struct Measurement measurement =  
{  
    .sensor_id = 17,  
    .value = 12.4,  
    .error = 0.2,  
    .timestamp = 1722258000  
};
```

Назначенные инициализаторы уменьшают риск ошибки.

Частичная инициализация

```
struct Measurement measurement =  
{  
    .value = 12.4,  
    .error = 0.2  
};
```

Остальные поля будут инициализированы нулями.

```
timestamp = 0  
sensor_id = 0
```

Нулевая инициализация

```
struct Measurement measurement = {0};
```

Все поля структуры получают нулевые значения.

**Это удобный способ получить структуру
в известном начальном состоянии.**

Массив структур

Вернёмся к исходной задаче.

Было:

```
double values[1000];  
double errors[1000];  
long timestamps[1000];  
int sensor_ids[1000];
```

Стало:

```
struct Measurement measurements[1000];
```

Один элемент — одно измерение

```
struct Measurement measurements[1000];
```

Теперь

```
measurements[i]
```

это одно измерение.

А его характеристики:

```
measurements[i].value  
measurements[i].error  
measurements[i].timestamp
```

Как читать выражение?

```
measurements[i].value
```

1. `measurements` — массив;
2. `measurements[i]` — один элемент массива;
3. этот элемент является структурой;
4. `.value` — поле этой структуры.

Инициализация массива структур

```
struct Measurement measurements[] =  
{  
    {  
        .value = 12.4,  
        .error = 0.2,  
        .timestamp = 1722258000,  
        .sensor_id = 17  
    },  
    {  
        .value = 8.7,  
        .error = 0.1,  
        .timestamp = 1722258060,  
        .sensor_id = 4  
    }  
};
```

Обход массива структур

```
for (int i = 0; i < count; ++i)
{
    printf(
        "%.1f ± %.1f, sensor %d\n",
        measurements[i].value,
        measurements[i].error,
        measurements[i].sensor_id
    );
}
```

Массив структур перебирается обычным циклом.

```

#include <stdio.h>

struct Measurement
{
    double value;
    double error;
    long timestamp;
    int sensor_id;
};

int main(void)
{
    struct Measurement m =
    {
        .value = 12.4,
        .error = 0.2,
        .timestamp = 1722258000,
        .sensor_id = 17
    };

    printf("%.1f ± %.1f\n", m.value, m.error);

    return 0;
}

```

Типичная ошибка

```
struct Measurement  
{  
    double value;  
    double error;  
};
```

```
Measurement m;    // ошибка
```

В языке **C** без **typedef** нужно писать:

```
struct Measurement m;
```

Типичная ошибка

```
struct Measurement
{
    double value;
    double error;
} // ошибка
```

После объявления структуры нужна точка с запятой.

```
struct Measurement
{
    double value;
    double error;
};
```

Типичная ошибка

```
struct Measurement measurement;  
  
printf("%f\n", measurement.value);
```

Поле `value` не было инициализировано.

Нельзя читать неинициализированные поля структуры.

Новый тип данных

```
struct Vector3  
{  
    double x;  
    double y;  
    double z;  
};
```

Теперь существует новый тип данных:

```
struct Vector3
```

Так же, как существуют `int`, `double`, `char`.

Объявление переменных

```
int count;
```

```
double temperature;
```

```
struct Vector3 velocity;
```

Во всех случаях создаётся переменная.

Различается только её тип.

Один тип — много объектов

```
struct Vector3 position;  
struct Vector3 velocity;  
struct Vector3 acceleration;
```

Все переменные имеют одинаковый тип: `struct Vector3`.

Но каждая хранит собственные значения.

Структуру можно передавать функции

```
void print_vector(struct Vector3 v)
{
    printf("(%.2f, %.2f, %.2f)\n",
           v.x, v.y, v.z);
}
```

Вызов:

```
print_vector(velocity);
```

Почему это удобно?

Без структуры пришлось бы писать:

```
void print_vector(  
    double x,  
    double y,  
    double z  
);
```

Со структурой:

```
void print_vector(  
    struct Vector3 v  
);
```

Параметр — обычная структура

```
void print_vector(struct Vector3 v)
{
    printf("%.2f\n", v.x);
}
```

Внутри функции `v` — обычная переменная типа `struct Vector3`.

Поэтому используется уже знакомый оператор:

```
v.x
v.y
v.z
```

Передача по значению

```
void reset(struct Vector3 v)
{
    v.x = 0;
    v.y = 0;
    v.z = 0;
}
```

```
reset(velocity);
```

После вызова `velocity` не изменится.

Функция получила собственную копию структуры.

Функция может вычислять результат

```
double length_squared(struct Vector3 v)
{
    return v.x * v.x +
           v.y * v.y +
           v.z * v.z;
}
```

Структуру можно использовать
как обычный параметр функции.

Функция может получать несколько структур

```
double dot_product(  
    struct Vector3 a,  
    struct Vector3 b  
)  
{  
    return  
        a.x * b.x +  
        a.y * b.y +  
        a.z * b.z;  
}
```

Функция получает два вектора
и возвращает число.

Функция может возвращать структуру

```
struct Vector3 make_vector(  
    double x,  
    double y,  
    double z  
)  
{  
    struct Vector3 result =  
    {  
        .x = x,  
        .y = y,  
        .z = z  
    };  
  
    return result;  
}
```

Использование результата

```
struct Vector3 velocity =  
    make_vector(  
        1.5,  
        -2.0,  
        0.5  
    );
```

Тип возвращаемого значения: `struct Vector3`

Сложение векторов

```
struct Vector3 add_vectors(  
    struct Vector3 a,  
    struct Vector3 b  
)  
{  
    return (struct Vector3)  
    {  
        .x = a.x + b.x,  
        .y = a.y + b.y,  
        .z = a.z + b.z  
    };  
}
```

Использование

```
struct Vector3 sum =  
    add_vectors(first, second);
```

Получается новое значение типа `struct Vector3`.

Структуры можно присваивать

```
struct Vector3 first =  
{  
    .x = 1,  
    .y = 2,  
    .z = 3  
};  
  
struct Vector3 second;  
  
second = first;
```

Копируются все поля структуры.

Один и тот же тип

```
struct Vector3 first;  
struct Vector3 second;  
  
second = first;
```

Присваивание возможно,
если обе переменные
имеют один и тот же *структурный тип*.

Одинаковые поля — не одинаковый тип

```
struct Position
{
    double x;
    double y;
    double z;
};
```

```
struct Velocity
{
    double x;
    double y;
    double z;
};
```

Ошибка

```
struct Position position;  
struct Velocity velocity;  
  
position = velocity;
```

Совпадение полей не делает структуры одним типом.

Полная программа

[Открыть пример в Compiler Explorer →](#)

Структуры внутри структур

До этого полями структуры были встроенные типы данных.

```
int  
double  
char
```

Но полем структуры может быть и другая структура.

Дата

```
struct Date {  
    int year;  
    int month;  
    int day;  
};
```

Дата — самостоятельный тип данных.

Событие

```
struct Event {  
    char name[64];  
  
    struct Date date;  
};
```

Поле `date` имеет тип `struct Date`.

Создание объекта

```
struct Event lecture = {  
    .name = "Programming",  
  
    .date =  
    {  
        .year = 2026,  
        .month = 9,  
        .day = 1  
    }  
};
```

Доступ к полям

```
lecture.date.year
```

```
lecture.date.month
```

```
lecture.date.day
```

Оператор `.` можно применять несколько раз подряд.

Более сложная структура

```
struct Time {  
    int hour;  
    int minute;  
    int second;  
};  
  
struct Event {  
    char name[64];  
  
    struct Date date;  
    struct Time time;  
};
```

Обращение к полям

```
lecture.date.year
```

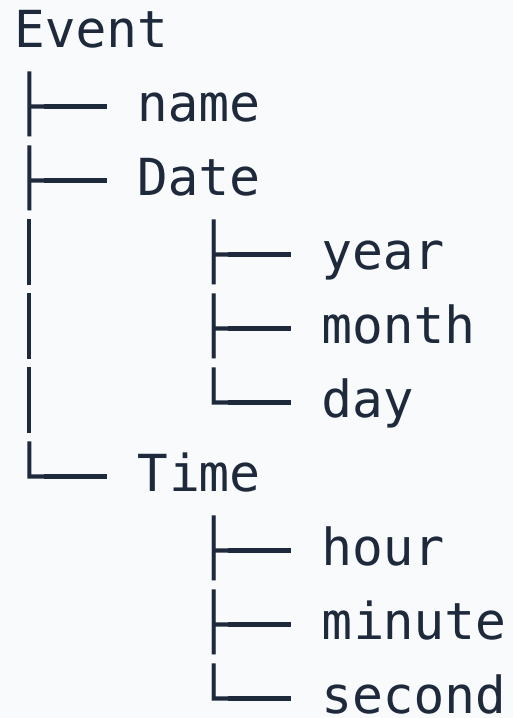
```
lecture.time.hour
```

```
lecture.time.minute
```

Каждый оператор `.` переходит на следующий уровень.

Композиция

Структуры можно собирать из других структур.



Другой пример

```
struct Point {  
    double x;  
    double y;  
};  
  
struct Rectangle {  
    struct Point top_left;  
    struct Point bottom_right;  
};
```

Обращение к координатам

```
rectangle.top_left.x
```

```
rectangle.top_left.y
```

```
rectangle.bottom_right.x
```

```
rectangle.bottom_right.y
```

Структуры можно присваивать

```
struct Date today = {  
    .year = 2026,  
    .month = 9,  
    .day = 1  
};
```

```
lecture.date = today;
```

Поле `date` имеет тип `struct Date`.

Когда нужен отдельный тип?

Если объект может использоваться самостоятельно — создайте отдельную структуру.

```
struct Date {  
    ...  
};  
  
struct Time {  
    ...  
};  
  
struct Address {  
    ...  
};
```

Иногда отдельный тип не нужен

```
struct Window {  
    char title[64];  
  
    struct {  
        int x;  
        int y;  
    } position;  
  
    struct {  
        int width;  
        int height;  
    } size;  
};
```

Когда так удобно?

Если группа полей используется только внутри одной структуры.

Например:

- положение окна;
- размер окна;
- параметры отображения.

Обращение к полям

```
window.position.x
```

```
window.position.y
```

```
window.size.width
```

```
window.size.height
```

Никакой новый синтаксис не появляется.

Что выбрать?

- Отдельная структура

```
struct Date {  
    ...  
};  
  
struct Event {  
    struct Date date;  
};
```

Если тип используется в нескольких местах программы.

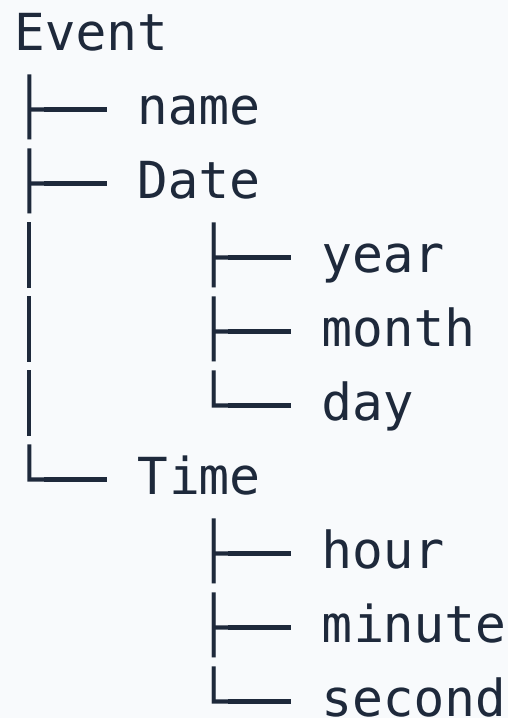
Что выбрать?

● Вложенная структура

```
struct Window {  
    struct  
    {  
        ...  
    } size;  
};
```

Если группа полей нужна только внутри одной структуры.

Хорошая структура отражает реальный объект



Структуры помогают описывать не набор отдельных переменных, а объекты предметной области.

Работа со структурами по адресу

До этого структуры передавались в функции по значению.

Это удобно, но не всегда эффективно.

Передача по значению

```
void print_student(struct Student student ) {  
    printf("%s\n", student.name);  
}
```

При вызове функции

```
print_student(student);
```

создаётся копия структуры.

Если структура маленькая

```
struct Student {  
    char name[64];  
    int group;  
    double average;  
};
```

Копирование такой структуры обычно не вызывает проблем.

Но структура может быть большой

```
struct Image {  
    char pixels[1000000];  
  
    int width;  
    int height;  
};
```

Не хочется копировать миллион байт при каждом вызове функции.

Решение

Передадим не саму структуру, а её адрес.

```
void print_student(struct Student *student );
```

Передача адреса

```
struct Student student = {  
    .name = "Alice",  
    .group = 101,  
    .average = 4.8  
};  
  
print_student(&student);
```

Оператор `&` получает адрес объекта.

Что изменилось?

Раньше

```
student
```

был структурой.

Теперь

```
student
```

внутри функции — указатель на структуру.

Доступ через указатель

Нельзя написать

```
student.name
```

Потому что `student` не является структурой.

Разыменование

Сначала нужно получить саму структуру.

```
(*student).name
```

```
(*student).group
```

```
(*student).average
```

Почему нужны скобки?

Правильно

```
(*student).name
```

Неправильно

```
*student.name
```

Оператор `.` имеет более высокий приоритет.

Сокращённая запись

В языке **C** существует специальный оператор

→

Он означает

получить структуру
и обратиться к её полю

Оператор **->**

Вместо

```
(*student).name
```

обычно пишут

```
student->name
```

Это полностью эквивалентные записи.

Использование

```
void print_student(struct Student *student) {  
    printf("%s\n",  
           student->name);  
  
    printf("%d\n",  
           student->group);  
  
    printf("%.2f\n",  
           student->average);  
}
```

Теперь функция может изменить объект

```
void reset_student(struct Student *student) {  
    student->average = 0.0;  
}
```

Изменение исходной структуры

```
reset_student(&student);
```

После вызова

```
student.average
```

станет равным

```
0.0
```

Потому что функция работала с исходным объектом.

Если изменять не нужно

```
void print_student(const struct Student *student) {  
    printf("%s\n",  
        student->name);  
}
```

Ключевое слово

const

запрещает изменять структуру через указатель.

Когда использовать const?

● Только чтение

```
const struct Student *student
```

Например

- печать;
- вычисления;
- поиск;
- сравнение.

Когда const не подходит?

Если функция должна изменить структуру.

```
void reset_student(struct Student *student) {  
    ...  
}
```

Массив структур

```
struct Student students[100];
```

Можно передать один элемент массива.

```
print_student(&students[i]);
```

Что получает функция?

students[i]



структура



&



адрес структуры



struct Student *

Самоссылочная структура

```
struct Node {  
    int value;  
  
    struct Node *next;  
};
```

Поле `next` хранит адрес следующего узла.

Почему не так?

```
struct Node {  
    int value;  
  
    struct Node next;  
};
```

✗ Невозможно.

Размер структуры нельзя определить.

Копирование структур

До этого мы уже использовали

```
second = first;
```

Но всегда ли такое копирование работает так, как мы ожидаем?

Простое копирование

```
struct Point {  
    int x;  
    int y;  
};  
  
struct Point first = {  
    .x = 10,  
    .y = 20  
};  
  
struct Point second;  
  
second = first;
```

Что получилось?

После выполнения

```
second = first;
```

first

x = 10

y = 20

second

x = 10

y = 20

Обе структуры содержат одинаковые значения.

Независимые объекты

```
second.x = 100;
```

```
first.x   = 10
```

```
second.x  = 100
```

Изменение одной структуры не влияет на другую.

А если внутри массив?

```
struct Student {  
    char name[64];  
    int group;  
};  
  
struct Student first = {  
    .name = "Alice",  
    .group = 101  
};  
  
struct Student second;  
  
second = first;
```

Что произошло?

После копирования

```
first.name = "Alice"
```

```
second.name = "Alice"
```

Но это две разные строки, расположенные в разных областях памяти.

Проверим

```
second.name[0] = 'M';
```

Получится

```
first.name == Alice
```

```
second.name === Mlice
```

Массив копируется полностью.

Теперь изменим структуру

```
struct Student {  
    char *name;  
    int group;  
};
```

Что теперь произойдёт при выполнении?

```
second = first;
```

До копирования

```
first
```

```
group = 101
```

```
name
```

```
|
```



```
"Alice"
```

После копирования

first

name —
|
▼
"Alice"

name —
|
▲

second

Скопировался указатель, а не строка.

Неожиданная проблема

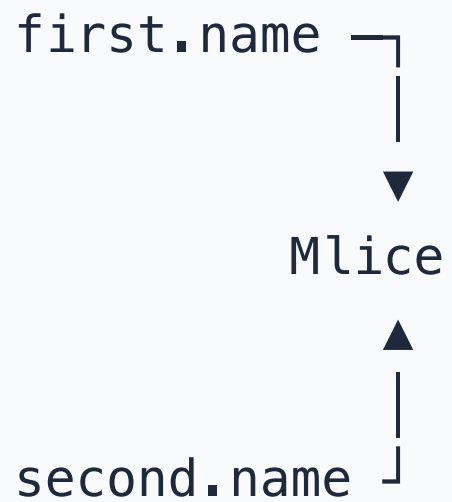
```
second.name[0] = 'M';
```

Что получится?

```
first.name == Mlice
```

```
second.name == Mlice
```

Почему?



Обе структуры используют одну строку.

Ещё одна проблема

```
free(second.name);
```

Что произошло?

Строка освобождена.

Но оба указателя по-прежнему содержат тот же адрес.

Опасность

После

```
free(second.name);
```

`first.name` → висячий указатель

`second.name` → висячий указатель

А если так?

```
free(first.name);
```

```
free(second.name);
```

Одна и та же память освобождается дважды.

Это серьёзная ошибка программы.

Как решить проблему?

Нужно создать новую строку и скопировать её содержимое.

Такое копирование называется **глубоким**.

После глубокого копирования

```
first.name -> "Alice"
```

```
second.name -> "Alice"
```

Теперь это две независимые строки.

Пример

```
struct Student copy;  
  
copy.group = original.group;  
  
copy.name = malloc(  
    strlen(original.name) + 1  
);  
  
strcpy(copy.name,  
        original.name);
```

Лучше оформить в функцию

```
struct Student  
copy_student(const struct Student *student);
```

Функция создаёт полностью независимую копию структуры.

Освобождение памяти

```
void destroy_student(struct Student *student) {  
    free(student->name);  
  
    student->name = NULL;  
}
```

Если структура владеет памятью, она должна уметь её освободить.

Что копирует оператор = ?

Поле	Результат
<code>int</code>	копируется значение
<code>double</code>	копируется значение
<code>char name[64]</code>	копируется весь массив
<code>struct Point</code>	копируется вся структура
<code>char *name</code>	копируется только адрес

Когда нужно глубокое копирование?

Поле	Deep copy
<code>int</code>	✗
<code>double</code>	✗
<code>char name[64]</code>	✗
<code>struct Point</code>	✗
<code>char *name</code>	✓
<code>int *values</code>	✓

Структуры в памяти

Структура в памяти — это не просто сумма размеров её полей.

Компилятор может добавлять дополнительные байты.

Эти байты называются **padding**.

Простой случай

```
struct Color {  
    unsigned char red;  
    unsigned char green;  
    unsigned char blue;  
};
```

```
printf("%zu\n", sizeof(struct Color));
```

Типичный результат:

3

Всё ожидаемо

red green blue

1 Б	1 Б	1 Б
-----	-----	-----

$1 + 1 + 1 = 3$ байта

Неожиданный пример

```
struct Example {  
    char symbol;  
    int value;  
};
```

```
printf("%zu\n", sizeof(struct Example));
```

Можно ожидать:

1 + 4 = 5 байт

Но часто результат будет:

8

Откуда появились лишние байты?

Смещение	Содержимое
0	symbol
1	padding
2	padding
3	padding
4	value
5	value
6	value
7	value

`value` начинается со смещения `4`.

Padding

Padding — это неиспользуемые байты, которые компилятор добавляет для правильного выравнивания данных.

symbol padding padding padding value

char				int
------	--	--	--	-----

Зачем нужно выравнивание?

Процессору обычно удобнее читать данные, если они расположены по подходящим адресам.

Например, `int` размером 4 байта часто удобнее хранить с адреса, кратного четырём.

Выровненный int

```
1004  1005  1006  1007  
└──────── int ─────────┘
```

Число начинается с адреса, кратного четырём.

Такой доступ обычно проще и быстрее для процессора.

Невыровненный int

```
1000  1001  1002  1003 | 1004  1005  1006  1007
      └────────── int ─────────┘
```

Число может пересекать границу удобного блока памяти.

На некоторых архитектурах такой доступ медленнее или требует специальной обработки.

Типичные требования выравнивания

Тип	Размер	Типичное выравнивание
char	1	1
short	2	2
int	4	4
double	8	8

Конкретные значения зависят от платформы и компилятора.

Поля не переставляются

```
struct Example {  
    char symbol;  
    int value;  
};
```

Компилятор сохраняет порядок полей:

symbol → value

Он не может незаметно поменять их местами.

Поэтому появляется padding

symbol

↓

нужно разместить int

↓

ближайшее подходящее смещение: 4

0	1	2	3	4
symbol	pad	pad	pad	value

Смещение поля

Смещение поля — это расстояние в байтах от начала структуры до начала этого поля.

```
struct Example {  
    char symbol;  
    int value;  
};
```

Типичные смещения:

```
symbol: 0  
value:  4
```

Макрос offsetof

Смещение поля можно узнать с помощью макроса `offsetof`.

```
#include <stddef.h>

printf("%zu\n", offsetof(struct Example, symbol));

printf("%zu\n", offsetof(struct Example, value));
```

Типичный результат:

```
0
4
```

Как выбирается место для поля?

Компилятор размещает поля по порядку.

Для каждого следующего поля он выбирает ближайшее подходящее смещение.

```
struct Data {  
    char a;  
    short b;  
    int c;  
};
```

Пример размещения

Смещение	Содержимое
0	a
1	padding
2–3	b
4–7	c

Размер структуры:

8 байт

Выравнивание самой структуры

Структура тоже имеет требование выравнивания.

Обычно оно равно наибольшему требованию выравнивания среди её полей.

```
struct Example {  
    char symbol;  
    int value;  
};
```

Если `int` выравнивается по 4 байтам, то и `struct Example` обычно начинается с адреса, кратного 4.

Padding в конце структуры

```
struct Another {  
    int value;  
    char symbol;  
};
```

Можно ожидать:

4 + 1 = 5 байт

Но типичный размер:

8 байт

Хвостовой padding

Смещение	Содержимое
0–3	value
4	symbol
5	padding
6	padding
7	padding

Padding может находиться не только между полями, но и в конце структуры.

Зачем padding в конце?

Рассмотрим массив структур.

```
struct Another array[3];
```

Элементы массива должны идти подряд:

```
array[0] array[1] array[2]
```

Каждый элемент должен начинаться с подходящего адреса.

Если бы размер был 5 байт

array[0] начинается с 1000

array[1] начинается с 1005

array[2] начинается с 1010

Адрес 1005 не кратен 4.

Поля `int` внутри следующих элементов оказались бы невыровненными.

Поэтому размер округляется

`array[0]` начинается с 1000

`array[1]` начинается с 1008

`array[2]` начинается с 1016

Расстояние между элементами массива равно

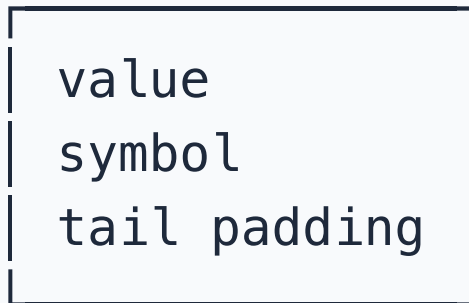
```
sizeof(struct Another)
```

Padding в массиве структур

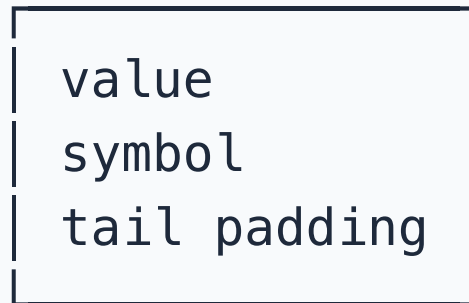
У массива нет отдельного padding между элементами.

Padding уже входит в размер каждого элемента.

array[0]



array[1]



Неудачный порядок полей

```
struct Bad {  
    char first;  
    int value;  
    char second;  
};
```

Типичное расположение:

0	first
1–3	padding
4–7	value
8	second
9–11	padding

Размер:

12 байт

Более компактный порядок

```
struct Better {  
    int value;  
    char first;  
    char second;  
};
```

Типичное расположение:

0–3	value
4	first
5	second
6–7	padding

Размер:

8 байт

Практическое правило

Часто выгодно располагать поля от более крупных к более мелким.

Например:

```
struct Record {  
    double measurement;  
    int sensor_id;  
    short status;  
    char category;  
    char valid;  
};
```

Так обычно уменьшается количество padding.

Но не всегда стоит переставлять поля

Порядок полей может быть важен:

- для читаемости;
- для совместимости с бинарным форматом;
- для сетевого протокола;
- для файлового формата;
- для внешней библиотеки.

Оптимизируйте порядок полей только тогда, когда это действительно нужно.

Что важно помнить?

`sizeof(struct T)` включает:

- размеры всех полей;
- padding между полями;
- padding в конце структуры.

Поэтому размер структуры может быть больше суммы размеров её полей.

sizeof не читает память

```
struct Student {  
    int id;  
    double grade;  
};  
  
printf("%zu\n", sizeof(struct Student));
```

Размер структуры известен компилятору заранее.

Оператор `sizeof` не обращается к памяти и не подсчитывает размер во время выполнения программы.

Что находится в padding?

Padding — обычные байты памяти.

Их содержимое не определено языком **C**.

```
symbol  
padding  
padding  
padding  
value
```

Не следует читать или сравнивать содержимое padding.

Осторожно с memcmp

```
struct Point {  
    char a;  
    int b;  
};
```

Даже если

```
a == a  
b == b
```

padding может отличаться.

Поэтому

```
memcmp(&x, &y, sizeof(struct Point))
```

не всегда подходит для сравнения структур.

Зачем нужен typedef?

До сих пор мы писали

```
struct Student first;  
struct Student second;  
struct Student third;
```

Почему слово `struct` приходится писать каждый раз?

Новый тип или новое имя?

Что делает эта запись?

```
typedef struct Student student_t;
```

Создаёт новый тип?

Или просто новое имя?

typedef

```
typedef struct Student student_t;
```

После этого можно писать

```
student_t first;  
student_t second;  
student_t third;
```

Вместо

```
struct Student first;
```

Что происходит?

```
typedef struct Student student_t;
```

Компилятор запоминает

student_t

↓

struct Student

Теперь оба имени обозначают один и тот же тип.

typedef не создаёт новый тип

```
typedef int Integer;
```

```
Integer a = 10;
```

```
int b = 20;
```

Переменные

Integer и int

имеют одинаковый тип.

Можно давать удобные имена

```
typedef unsigned char Byte;  
  
typedef unsigned long ulong;
```

Теперь можно писать

```
Byte data;  
  
ulong size;
```

typedef для структуры

До

```
struct Student student;
```

После

```
typedef struct Student student_t;  
  
student_t student;
```

Часто используют такую запись

```
typedef struct {  
    char name[64];  
    int age;  
}  
student_t;
```

Имя структуры здесь отсутствует.

Остаётся только имя типа:

```
student_t
```

typedef для массивов

Без typedef

```
int matrix[3][3];
```

С typedef

```
typedef int Matrix3x3[3][3];
```

```
Matrix3x3 matrix;
```

typedef для указателей

```
typedef int *IntPtr;
```

```
IntPtr p;
```

Эквивалентно

```
int *p;
```

Частая ошибка

```
typedef int *IntPtr;
```

```
IntPtr a, b;
```

Что получится?

a

b

Ответ

Получится

```
int *a;  
int *b;
```

Типом является

IntPtr

а не

*

Когда typedef полезен?

✓ длинные объявления

```
struct Student
```

✓ сложные типы

```
int Matrix3x3[3][3];
```

✓ пользовательские типы

```
Student
```

Когда typedef ухудшает код?

Плохо

```
typedef unsigned long long ULL;
```

```
typedef unsigned char UC;
```

```
typedef unsigned short US;
```

Такие сокращения обычно делают код менее понятным.

Перечисления (enum)

Не каждая переменная может принимать любое значение.

Например, возраст может быть любым числом, а день недели — только одним из семи вариантов.

Магические числа

```
int direction = 2;  
  
if (direction == 3) {  
    printf("Turn left\n");  
}
```

Что означают числа

2
3

Из программы это непонятно.

Можно придумать понятные имена

Вместо

0

1

2

3

хочется писать

NORTH

SOUTH

WEST

EAST

enum

```
enum Direction {  
    NORTH,  
    SOUTH,  
    WEST,  
    EAST  
};
```

Перечисление объединяет фиксированный набор именованных значений.

Переменная типа enum

```
enum Direction direction;  
  
direction = EAST;
```

Теперь переменная может хранить направление.

Код читается значительно лучше.

Использование

```
if (direction == NORTH) {  
    printf("Move forward\n");  
} else {  
    printf("Turn\n");  
}
```

Названия сами объясняют смысл программы.

Что хранится в памяти?

Хотя мы пишем

```
NORTH  
SOUTH  
WEST  
EAST
```

компилятор хранит целые числа. Типично:

```
NORTH = 0  
SOUTH = 1  
WEST  = 2  
EAST  = 3
```

Значения по умолчанию

```
enum Direction {  
    NORTH,  
    SOUTH,  
    WEST,  
    EAST  
};
```

Если значения не указаны явно,
они автоматически получают

```
0, 1, 2, 3, ...
```

Явные значения

```
enum HttpStatus {  
    OK = 200,  
    NOT_FOUND = 404,  
    SERVER_ERROR = 500  
};
```

При необходимости можно задать значения самостоятельно.

Следующее значение

```
enum Example {  
    FIRST = 10,  
    SECOND,  
    THIRD  
};
```

Получится

```
FIRST   = 10  
SECOND  = 11  
THIRD   = 12
```

Не все значения обязательно задавать

```
enum Example {  
    A = 5,  
    B,  
    C = 20,  
    D  
};
```

Получится

```
A = 5  
B = 6  
C = 20  
D = 21
```

enum и switch

```
switch (direction) {  
case NORTH:  
    printf("North\n");  
    break;  
  
case SOUTH:  
    printf("South\n");  
    break;  
  
case WEST:  
    printf("West\n");  
    break;  
  
case EAST:  
    printf("East\n");  
    break;  
}
```

Где удобно использовать enum?

```
enum Weekday
```

```
enum TrafficLight
```

```
enum Direction
```

```
enum ErrorCode
```

Во всех случаях количество возможных значений заранее известно.

Ограничение enum

```
enum Direction direction;  
  
direction = 100;
```

Такой код обычно компилируется.

enum делает программу понятнее, но сам по себе не запрещает другие целочисленные значения.

enum и структуры

```
struct Robot {  
    int x;  
    int y;  
  
    enum Direction direction;  
};
```

Структура описывает объект, а перечисление — одно из его возможных состояний.

Что важно запомнить?

`enum` создаёт тип, значениями которого являются именованные константы.

В памяти перечисления обычно хранятся как целые числа.

Объединения (union)

Структура хранит все свои поля одновременно.

Но иногда объекту нужно хранить только одно из нескольких возможных значений.

В памяти



Память выделяется для каждого поля.

Но всегда ли это нужно?

Представим калькулятор.

Результат вычисления может быть

целым числом

или

вещественным числом

Но не обоими одновременно.

Лишняя память

Если использовать структуру, память выделяется сразу для всех полей.

Хотя одновременно используется только одно из них.

union

```
union Value {  
    int integer;  
    double real;  
};
```

Все поля используют **одну и ту же область памяти**.

В памяти



Поля полностью накладываются друг на друга.

Размер union

```
union Value {  
    int integer;  
    double real;  
};
```

```
printf("%zu\n", sizeof(union Value));
```

Размер объединения равен размеру **самого большого поля**.

Пример

`int` → 4 байта

`double` → 8 байт

Поэтому

```
sizeof(union Value)
```

↓

8 байт

Запись значения

```
union Value value;  
  
value.integer = 42;
```

Теперь память содержит представление целого числа.

Затем

```
value.real = 3.14;
```

Предыдущее значение перезаписывается.

Теперь память содержит представление числа с плавающей точкой.

Одновременно хранить нельзя

После записи

```
value.real = 3.14;
```

читать

```
value.integer
```

нельзя.

Последним записанным считается только одно поле.

Откуда программа знает тип?

Само объединение не хранит информацию, какое поле сейчас используется.

Эту информацию нужно хранить отдельно.

enum + union

```
enum ValueType {  
    INTEGER,  
    REAL  
};
```

Перечисление хранит тип текущего значения.

struct + enum + union

```
struct Variant {  
    enum ValueType type;  
  
    union {  
        int integer;  
        double real;  
    } value;  
};
```

Теперь известно, какое поле объединения содержит корректные данные.

Использование

```
if (variant.type == INTEGER) {  
    printf("%d\n", variant.value.integer);  
} else {  
    printf("%f\n", variant.value.real);  
}
```

Перед чтением проверяется тип значения.

Где используются union?

- интерпретаторы;
- компиляторы;
- сетевые протоколы;
- файловые форматы;
- драйверы устройств;
- операционные системы.

Везде, где объект может иметь несколько различных представлений.

Что важно запомнить?

`union` объединяет несколько полей, используя одну общую область памяти.

В каждый момент времени корректным считается только одно поле.

struct и union

struct	union
Каждое поле имеет собственную память	Все поля используют одну память
Все поля существуют одновременно	Одновременно используется только одно поле
Размер — сумма размеров полей (с padding)	Размер — размер самого большого поля

Битовые поля

Иногда программе нужно хранить много небольших значений.

Например, логические признаки или небольшие числа.

Выделять для каждого целый байт не всегда эффективно.

Пример

Настройки игры.

Музыка — Включена?

Полный экран — Да / Нет

Показывать FPS — Да / Нет

Каждому признаку достаточно одного бита.

Обычная структура

```
struct Settings {  
    int music;  
    int fullscreen;  
    int fps;  
    int vsync;  
};
```

Каждое поле занимает отдельное место в памяти.

Но информации очень мало

Каждое поле хранит только два значения.

0

или

1

Получается, для одного бита информации используется целое число.

Битовое поле

```
struct Settings {  
    unsigned music      : 1;  
    unsigned fullscreen : 1;  
    unsigned fps        : 1;  
    unsigned vsync      : 1;  
};
```

После двоеточия указывается ширина поля в битах.

Использование

```
settings.music = 1;  
  
settings.fullscreen = 0;  
  
if (settings.fps) {  
    printf("Show FPS\n");  
}
```

Используются так же, как обычные поля структуры.

Ширина поля

```
struct Date {  
    unsigned day    : 5;  
    unsigned month  : 4;  
    unsigned year   : 12;  
};
```

Можно задавать любое количество бит, если его достаточно для хранения значения.

Как располагаются поля?

Можно представить память так.



Всего

$$5 + 4 + 12 = 21 \text{ бит}$$

Значит ли это,

что структура занимает 21 бит?

Не обязательно.

Компилятор учитывает правила выравнивания.

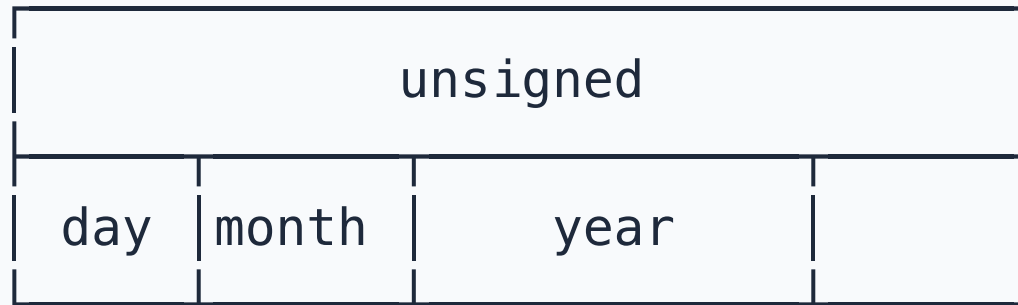
Контейнер

Обычно компилятор размещает несколько битовых полей внутри одного объекта базового типа.

Например,

`unsigned`

Типичное размещение



Свободные биты могут остаться неиспользованными.

Если места не хватает

```
struct Example {  
    unsigned a : 30;  
    unsigned b : 10;  
};
```

Обычно для следующего поля начинается новый контейнер.

Что гарантирует стандарт?

- ✓ Поля располагаются в порядке объявления.
- ✗ Не определяется, в каком именно бите будет лежать каждое поле.

Что зависит

от реализации?

- порядок битов;
- размер контейнера;
- выравнивание;
- упаковка полей;
- переход через границы контейнера.

Разные компиляторы могут работать по-разному.

Адрес битового поля

```
&settings.music
```

Так делать нельзя.

Битовое поле не является отдельным объектом памяти.

sizeof

```
sizeof(settings.music)
```

Такой код также недопустим.

Битовое поле не существует как самостоятельный объект.

Когда использовать?

Битовые поля подходят, если нужно

- экономить память;
- хранить множество флагов;
- описывать аппаратные регистры.

Не рекомендуется использовать их для переносимых двоичных форматов.

Что важно запомнить?

Битовое поле — поле структуры, размер которого задается в битах.

Компилятор старается разместить несколько битовых полей максимально компактно.

Обычные и битовые поля

Обычные поля	Битовые поля
Размер задается типом	Размер задается в битах
Имеют собственный адрес	Собственного адреса нет
Полностью определены стандартом	Многие детали зависят от реализации
Просты в использовании	Позволяют экономить память